

Revisiting the meaning of `foo(ConceptName, ConceptName)`

Document number: P0464R1

Authors: Tony van Eerd <tvaneerd@gmail.com>
Botond Ballo <botond@mozilla.com>

Date: 2016-11-08

Audience: Evolution Working Group

Abstract

In the current [Concepts Technical Specification](#)^[1], `R foo(ConceptName, ConceptName);` denotes a function that takes two arguments of the same type, with that type satisfying the concept `ConceptName`. More precisely, it's a shorthand for:

```
template <ConceptName __C>
R foo(__C a, __C b);
```

This paper argues that it would be more natural for it to denote a function that takes two arguments of potentially different types, with those types satisfying the concept `ConceptName`. That is, it should be a shorthand for:

```
template <ConceptName __C1, ConceptName __C2>
R foo(__C1 a, __C2 b);
```

Revision History

- **R1**
 - New sections under "Arguments": "Variadic templates" and "Perfect forwarding"
 - New section under "Counter-arguments": "Interaction with definition checking"
 - New section "Impact on the return type"
- **R0**
 - Initial draft

Arguments

Follows from first principles

Once a developer has learned the basic language rule in Concepts that `ConceptName var` means that the variable `var` must have some type that models the concept `ConceptName`, the meaning of `ConceptName a, ConceptName b` should be obvious: that `a` and `b` model `ConceptName` — no more, no less.

It is surprising for the actual meaning to be "`a` and `b` model `ConceptName` **and** they have the same type". If this additional constraint is desired, it should be stated in the code.

In other words, the interpretation this paper argues for is the one that follows from first principles. The authors believe this should be the guiding consideration.

Consistency with `auto`

`auto`, as standardized in C++14, already behaves the way we desire. That is,

```
R foo(auto a, auto b);
```

is a shorthand for:

```
template <typename __C1, typename __C2>
R foo(__C1 a, __C2 b);
```

Since `auto` can be thought of as the weakest concept, it would make the language simpler and easier to teach if the way concepts

behave is consistent with the way `auto` behaves.

Consistency with local variables

The current "same type" rule does not extend to local variables declared in the body of the function:

```
// a and b must have the same type, but var is allowed to be a different type
R foo(ConceptName a, ConceptName b) {
    ConceptName var = /* ... */;
}
```

It is very confusing to for two of the three uses of `ConceptName` in this piece of code to have an additional constraint between them, but not the third.

Variadic templates

The current behaviour is also inconsistent with the behaviour of variadic templates:

```
R foo(ConceptName... args); // args can have different types
```

This means that the property that a variadic template behaves as if you had written a bunch of overloads with different numbers of parameters, does not hold when the parameter types are specified by a concept name.

It also means that the common pattern of writing your variadic template like so:

```
R foo(ConceptName arg); // handle one argument (base case)
R foo(ConceptName arg1, ConceptName arg2, ConceptName... rest); // handle two or more arguments
```

results in a big surprise: `arg1` and `arg2` must have the same type, while the remaining arguments may have different types!

Concepts are to types as types are to values

Concepts are to types as types are to values, in the sense that a concept defines a set of valid types much like a type defines a set of valid values.

When we write `int x, int y`, `int` is a *type*, and the meaning is that `x` and `y` both have *type* `int`; beyond that, `x` and `y` are not required to have the same *value*.

Similar, when we write `ConceptName x, ConceptName y`, `ConceptName` is a *concept*, and the meaning is that `x` and `y` both model the *concept* `ConceptName`; beyond that, `x` and `y` should not be required to have the same *type*.

Iterator pairs are going out of fashion

One of the main arguments given for the current semantics is that having two arguments of the same (templated) type is very common in the standard library, due to iterator pairs.

However, in a modern C++ world, this argument does not hold water. Thanks to the [Ranges Technical Specification](#)^[2], iterator pairs are going out of fashion, being replaced with iterator/sentinel pairs — which are two potentially *different* types — and single range objects.

User confusion

Users have repeatedly expressed confusion about the current behaviour, indicating that it is not intuitive. Some examples:

- [Concepts mailing list discussion](#)^[3]
- [std-proposals discussion](#)^[4]
- [Reddit question](#)^[5]

Perfect forwarding

The "same type" requirement can be overly restrictive in situations involving perfect forwarding. Consider:

```
void foo(Container&& a, Container&& b);
// ...
std::vector<int> a;
foo(a, std::vector<int>{1, 2, 3}); // error: cannot deduce 'Container'
```

Here, even though the arguments have the same type, they have different value categories, and as a result, due to the use of perfect forwarding, the specific parameter types differ by a reference, preventing deduction.

How would we express the old meaning?

What if one wants to express the old meaning of `foo(ConceptName, ConceptName)`, a function with two parameters of the same type that satisfies a concept? In addition to the non-terse notation:

```
template <ConceptName C>
R foo(C a, C b);
```

one could also write:

```
R foo(ConceptName a, decltype(a) b);
```

A third alternative would be provided by the syntax proposed in [N3878](#)^[6]:

```
R foo(ConceptName{C} a, C b);
```

Granted, the second two forms are not exactly equivalent, because the second parameter does not participate in deduction. One can envision a variation of the syntax proposed in N3878 which would mean "same type, and treated equally for deduction":

```
R foo(ConceptName{C} a, ConceptName{C} b);
```

In any case, using notation available today, if equal treatment for deduction matters, the non-terse notation can be used.

Counter-arguments

Frequency of use

It can be argued that you don't often want a function that takes two arguments of potentially different types satisfying the same concept without having an additional relationship between the two types.

The authors believe that arguments like this based on frequency of use, should take a back seat to the arguments listed above, notably the argument based on first principles.

However, even from a frequency point of view, it should be noted that a [survey of standard library functions](#)^[7] found a significant amount of "same concept, different type" functions, comparable to the amount of "same concept, same type" functions when controlling for iterator pairs.

Interaction with definition checking

It can also be argued that this change will encourage template authors to write under-constrained templates, because they will opt to use the terse `R foo(ConceptName, ConceptName);` form even in cases where in there should be an additional constraint on the parameter types. A proliferation of under-constrained templates will make the introduction of definition checking harder, because an under-constrained template will not pass definition checking.

The authors acknowledge that this is a concern, but believe that the problem of under-constrained templates is not specific to this change. Template authors will sometimes write under-constrained templates without this change, too, such as by writing `R foo(Concept1, Concept2)` when an additional relationship between the types is present.

Chances are that, due to the presence of under-constrained templates (irrespective of this change), definition checking will be a opt-in feature anyways.

Impact on the return type

The case where a concept name appears both in a parameter type and a return type also needs to be addressed.

Currently, the following code:

```
ConceptName foo(ConceptName arg);
```

is a shorthand for:

```
template <ConceptName __C>
__C foo(__C arg);
```

With this proposal, for consistency, it ought to become a shorthand for:

```
template <ConceptName __C>
ConceptName foo(__C arg);
```

That is, the return type changes from being determined by the parameter type, to being deduced from the types of return expressions (while still being constrained by `ConceptName`). This has the implication that if the return type isn't deducible (e.g. because different return expressions have different types, or one return expression is a braced-init-list (as in `return { exprs };`)), the function becomes ill-formed.

If the original semantics is desired, the non-terse notation can be used:

```
template <ConceptName C>
C foo(C arg);
```

Alternatively, using the syntax proposed in N3878:

```
auto foo(ConceptName{C} arg) -> C;
```

Acknowledgements

The authors would like to thank Andrew Sutton, Guillaume Racicot, and everyone else who participated in discussions on this subject (on the public mailing lists, in private email correspondence, and elsewhere), for bringing a variety of valuable perspectives and arguments to the table.

References

1. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4553.pdf>
2. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4569.pdf>
3. https://groups.google.com/a/isocpp.org/d/topic/concepts/BFmvN_w-PEs/discussion
4. https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/_YpRok7LaEU/discussion
5. https://www.reddit.com/r/cpp/comments/53i9a2/conceptlite_short_notation_problem/
6. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3878.pdf>
7. <https://groups.google.com/a/isocpp.org/forum/#!topic/concepts/cdX4X0ngJqc/discussion>